《计组I》

第五章　处理器设计

李瑞　　教授

蒋志平　副教授

计算机科学与技术学院

温故 —关于指令系统

- 简述mov, lea, mov offset的关系?
- 简述串操作所需的7个要素
- 简述JMP和其它条件跳转指令
- 汇编中实现循环的典型方案?
- 汇编中如何定义“变量”?
- 简述procedure，如何调用procedure?
- 简述marco，如何调用marco?

回顾一下指令系统一章学了啥?

- 我们知道了CPU的逻辑结构：
  - CPU里面有通用寄存器，A/B/C/DX
  - 有ALU
  - 有段/偏移寄存器 CS:IP / DS:SI/DI/BX / SS:SP / ES
  - 有标志位Flags (SF, CF, OF, IF, PF, ZF …)
  - CPU里面用总线方式连接这些部件…

回顾一下指令系统一章学了啥?

- 我们还知道了:
  - 汇编其实是非常“简单”的语言...
  - 汇编/机器程序就是“流水帐”，数据与指令无区分
  - 运行时的程序在内存里，CPU从内存读指令执行
  - CPU运行哪里的代码?
    - CS:IP哪到哪就运行哪!
  - 学习+使用了一些常见的指令(mov, add, int, halt, ret...)
  - ...

回顾一下指令系统一章学了啥?

- 我们还知道了:
  - 多种寻址方式(为什么叫寻址?)
    - 指令的寻址方式（哪几种? ）
    - 数据的寻址方式（哪几种? ）
  - ...

回顾一下指令系统一章学了啥?

- 似乎，我们理解了...
  - CPU是基于寄存器器
  - CPU是基于地址运行的
  - CPU可以对一些指令正确响应&工作
  - 基于汇编语言的CPU操作方法
- 但是，
  - CPU到底是什么样?
  - 指令系统怎么设计的?
  - CPU是如何分解并执行指令的?
  - 以及最神秘的~“CPU内部的CPU”
    - CPU内部的微指令/操作如何实现

回顾一下指令系统一章学了啥?

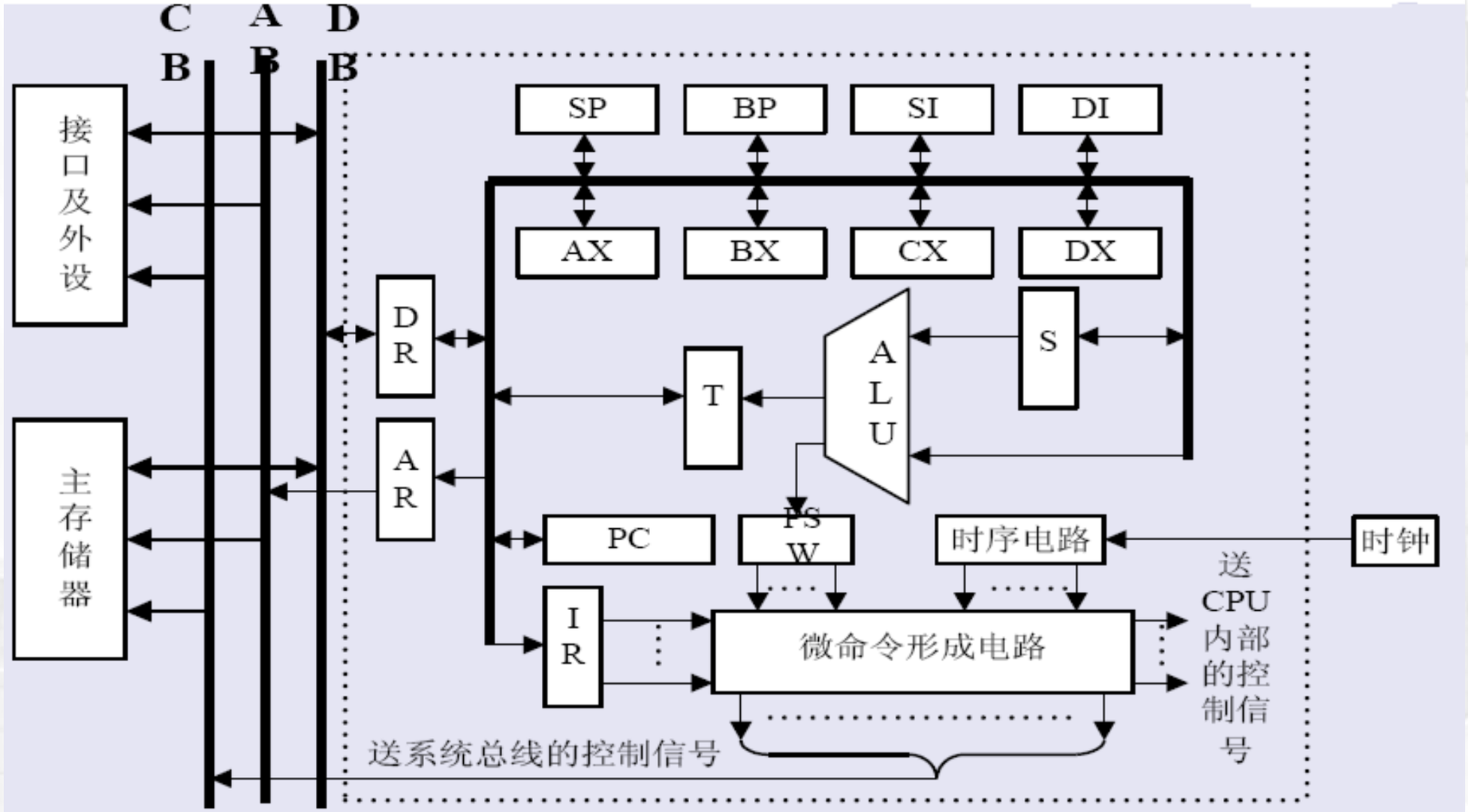
- 但是,
  - CPU到底是什么样?
  - 指令系统怎么设计的?
  - CPU是如何分解并执行指令的?
  - 以及最神秘的~“CPU内部的CPU”
    - CPU内部的微指令/操作如何实现

- 第五章 CPU设计
  - 5.1 CPU结构
  - 5.2 CPU指令设计
  - 5.3 指令执行过程
  - 5.4 CPU部件设计
    - 5.4.2 控制器的设计
      - 组合逻辑控制器
      - 微指令控制器
        - 微指令编码设计

CPU基本组成

CPU基本组成

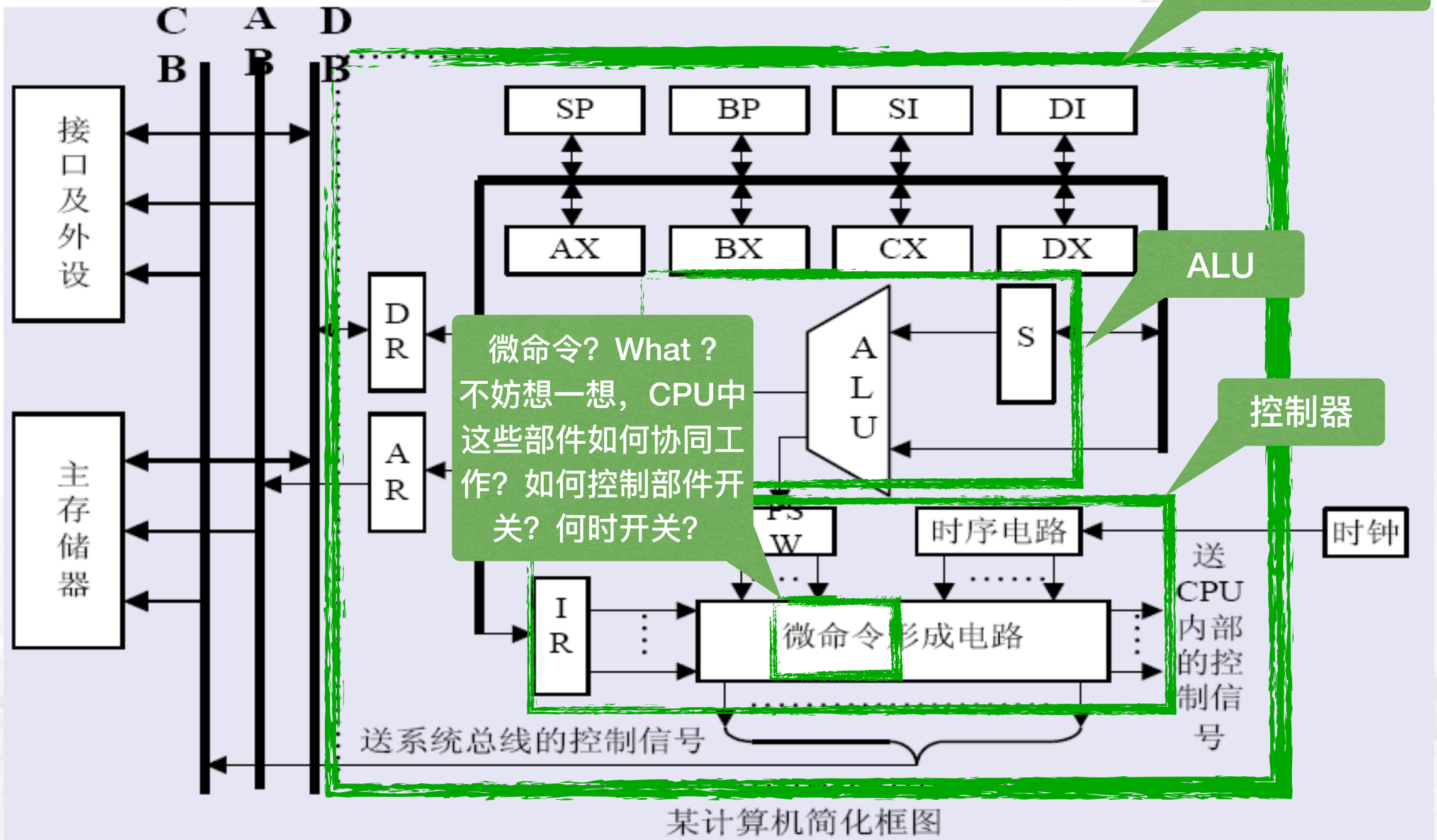
1. 部件：寄存器、ALU、控制器、其它
2. 数据通路(DataPath)：总线(BUS)、专用通路
3. 外部交互：外部总线



某计算机简化框图

CPU基本组成

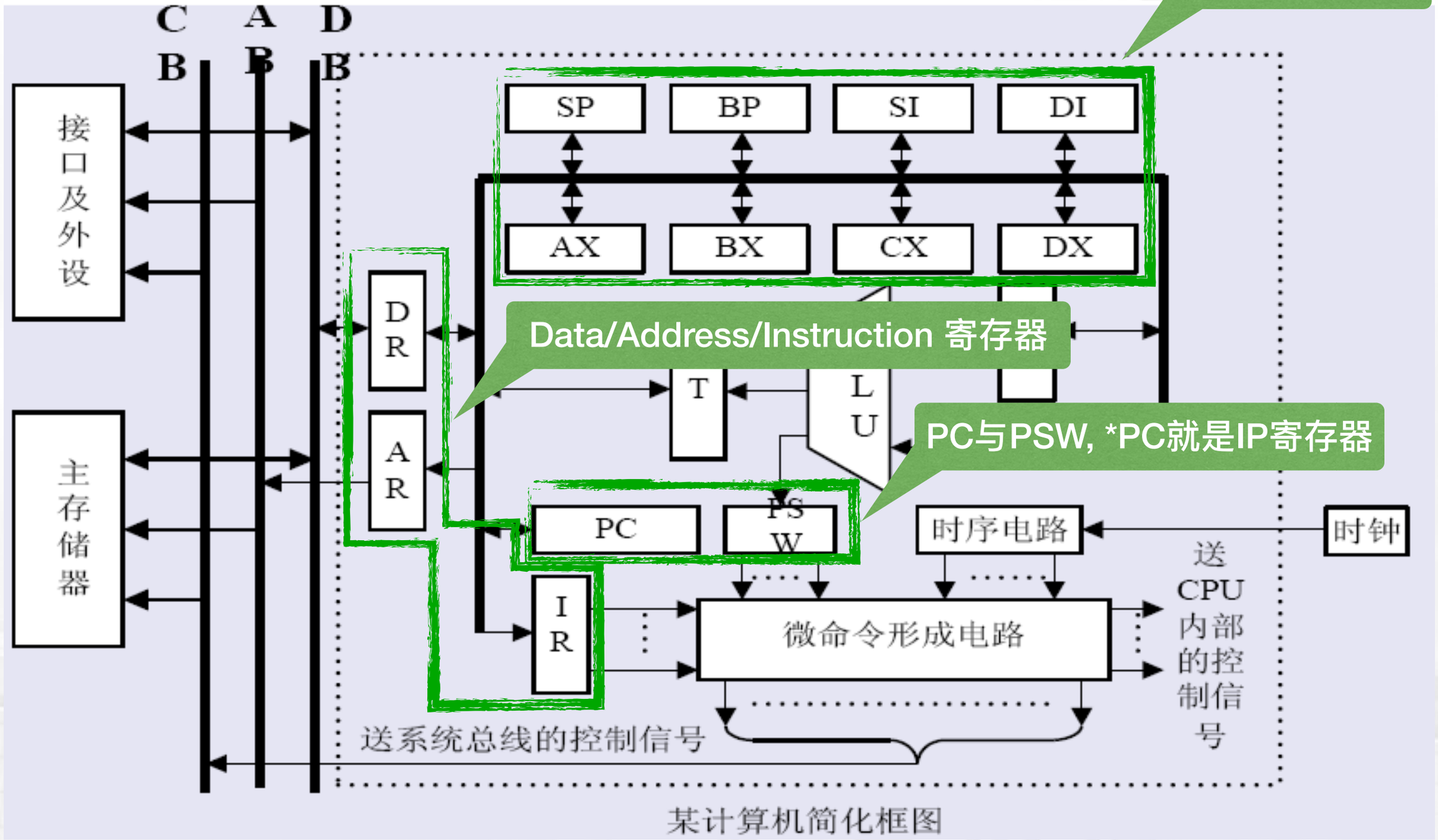
1. 部件：寄存器、ALU、控制器、其它



某计算机简化框图

CPU基本组成

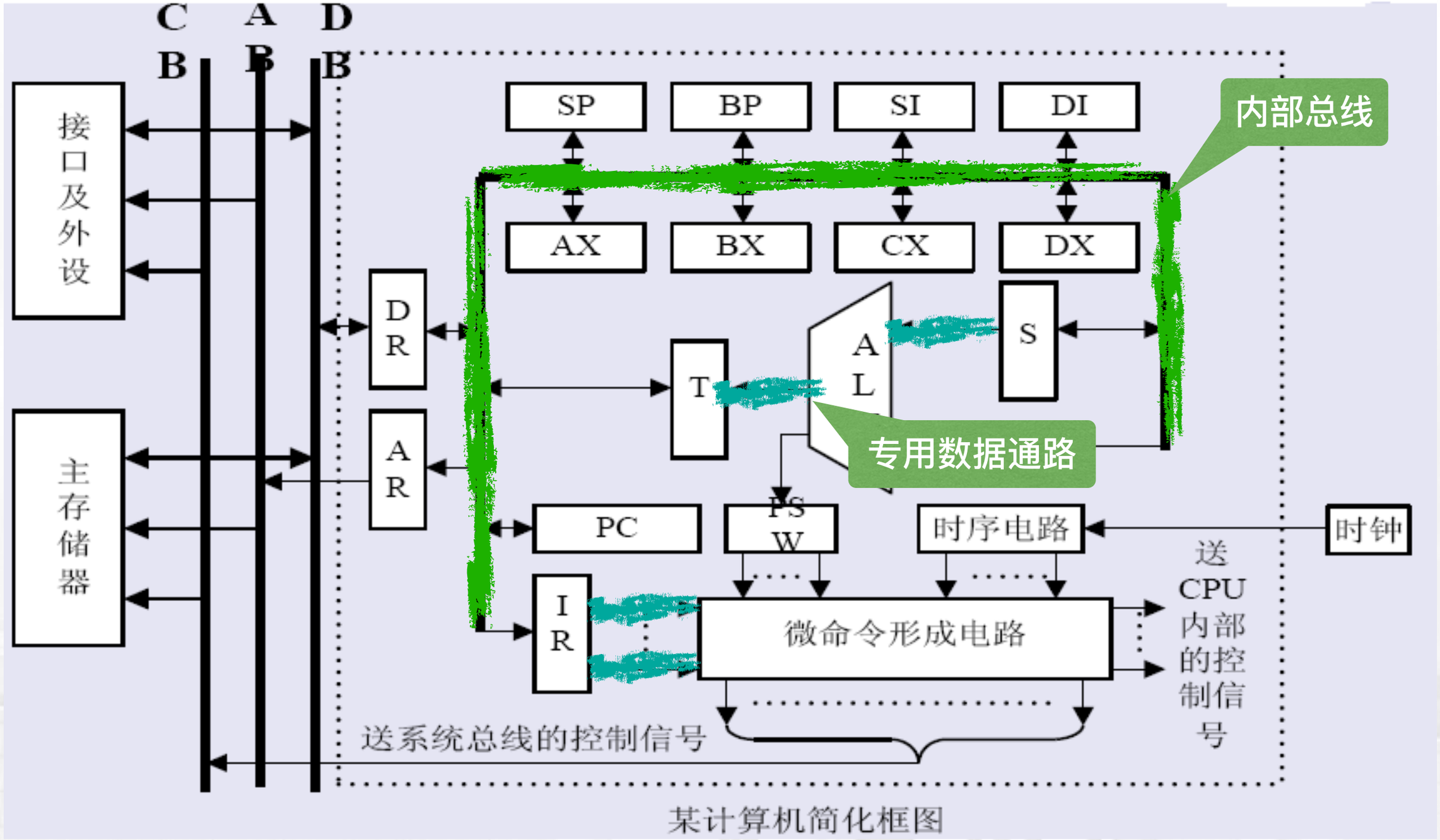
1. 部件：寄存器、ALU、控制器、其它



某计算机简化框图

CPU基本组成

2. 数据通路(DataPath)：总线(BUS)、专用通路

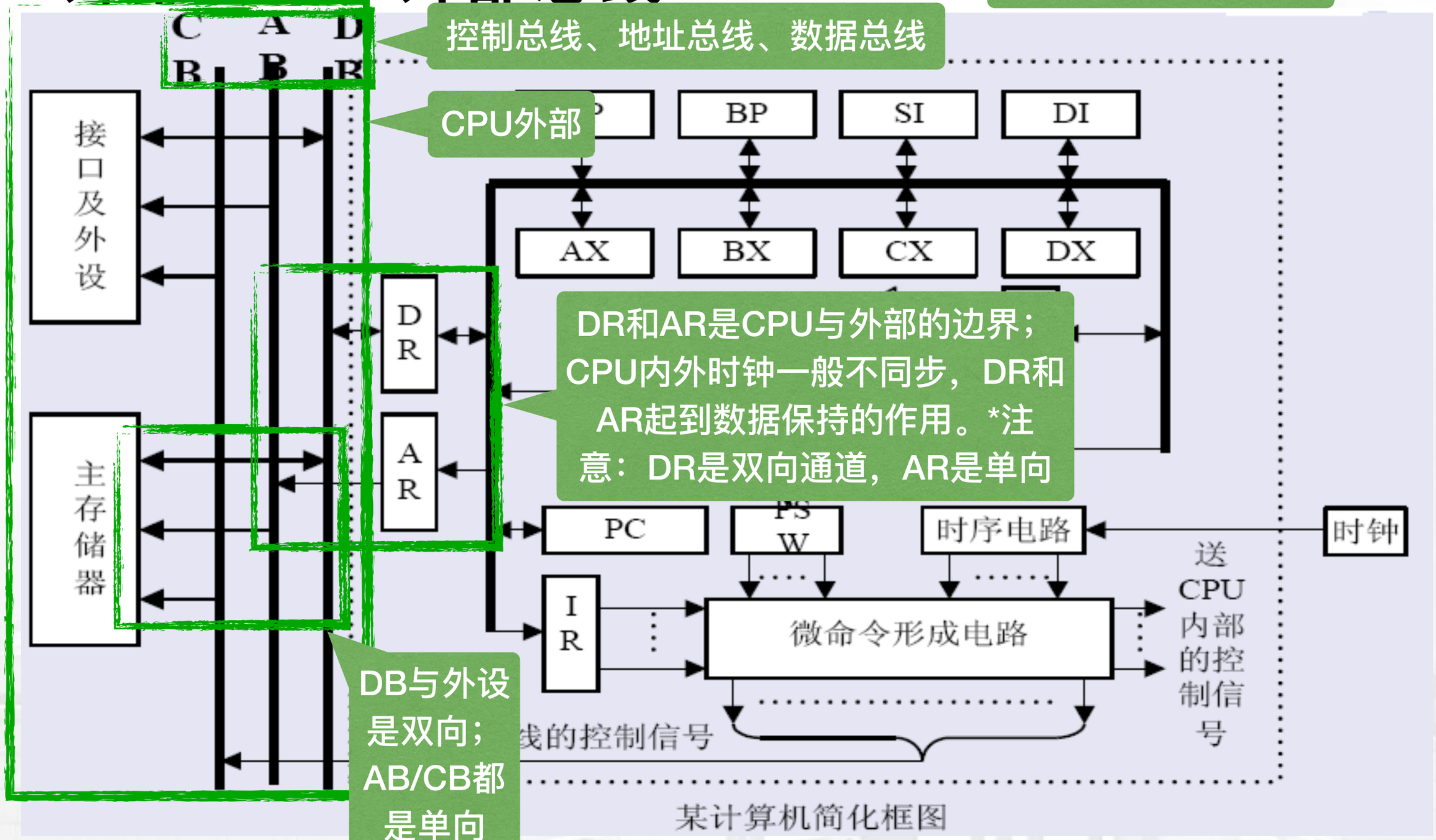


某计算机简化框图

CPU基本组成

3. 外部交互：外部总线

*想一想：3个总线、DR/AR/IR各多少位?



某计算机简化框图

CPU基本组成

- 数据总线和CPU内部总线是双向的。
- 总线的条数称为数据总线宽度。比如，16位总线，指其数据总线为16根。
- 数据总线是三态的，是指0，1和高阻抗三个状态。由于总线是公共通道，在某一时刻，只允许接收某一设备的信号，其他一切设备都应和它断开。

CPU基本组成

- 控制总线上传送一个部件对另一个部件的控制信号。
- 主设备与从设备:在总线上所连接的各类设备，按其对总线有无控制功能可分为主设备和从设备。 主设备对总线有控制权，从设备只能响应。
- 三态？不一定~

CPU基本组成

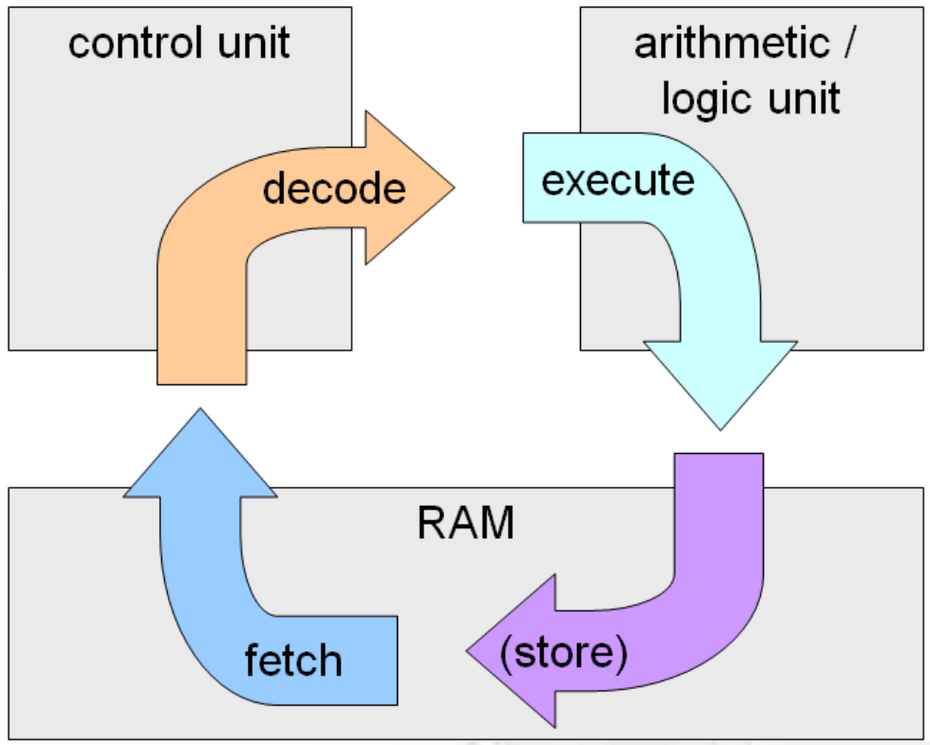
- 地址总线上传送地址信号，用来指定需要访问的地址/部件
- 总线上的所有部件接收到该地址信号，但只有经过译码电路选中的部件才接收控制信号。
- 地址总线是单向的
- 地址总线也是三态的

深入CPU指令执行过程

深入CPU指令执行过程

1、指令执行的基本过程

- 取指、执行
- 取指、取操作数、执行
- 取指、取操作数、执行、写结果



以指令add [2000h], ax为例，看看4个阶段的CPU内部执行流程

深入CPU指令执行过程

以指令add [2000h], ax为例

- 取指令Fetch
  - PC->AR
  - AR->AB
  - AB->MAR
  - Read (MM->MDR)
  - MDR->DB
  - DB->DR
  - DR->IR
  - PC++

CS:IP指向下一条指令的地址

内存里的AR

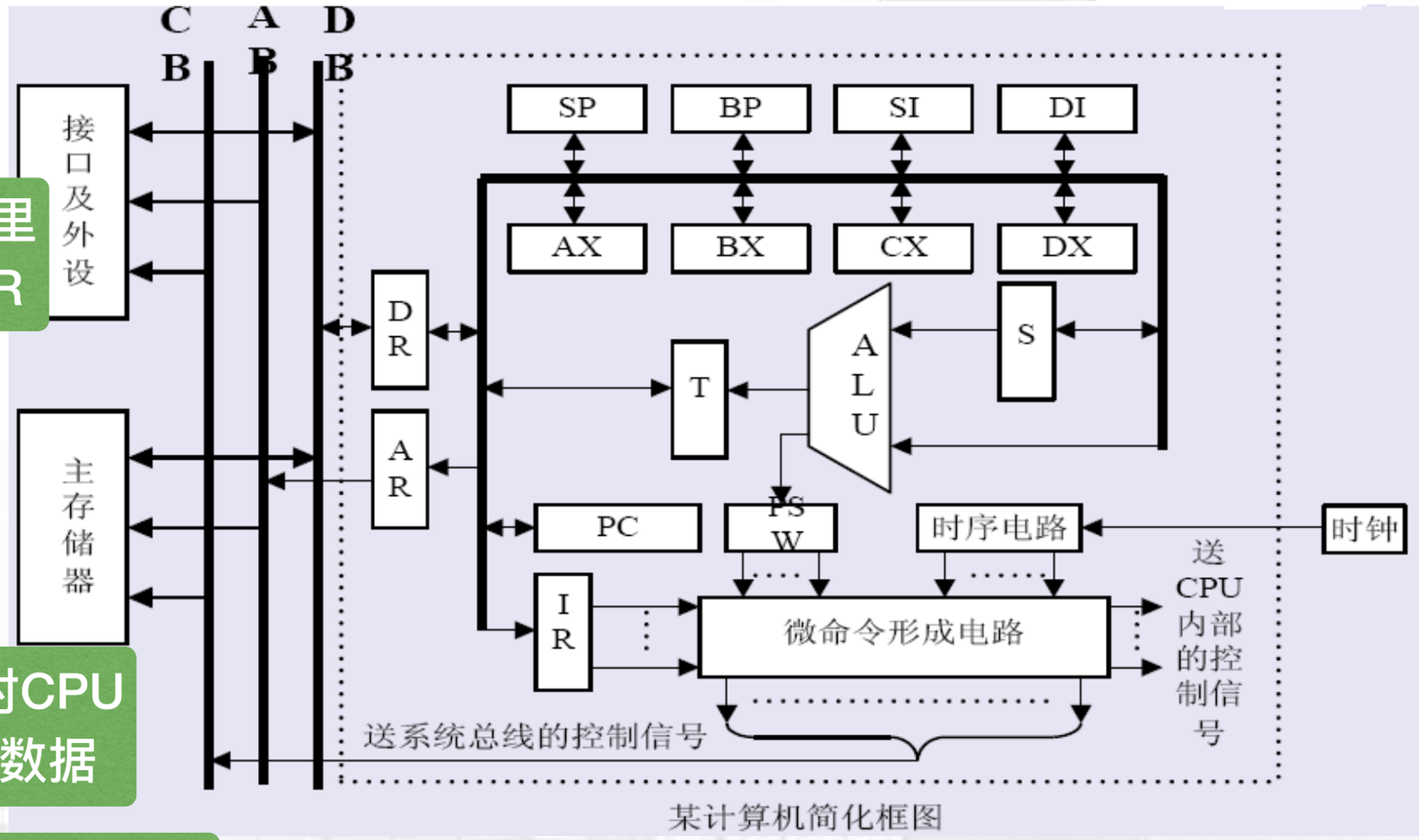
内存里的DR

内存

取出的指令对CPU来说，就是数据

把“指令数据”导出IR，才真的是看作指令

IP指针自加1



某计算机简化框图

深入CPU指令执行过程

以指令add [2000h], ax为例

- 取操作数Fetch Operands
  - PC->AR
  - AR->AB
  - AB->MAR
  - Read (MM->MDR)
  - MDR->DB
  - DB->DR
  - DR->S
  - PC++

CS:IP指向该指令的第二个字,[2000h]

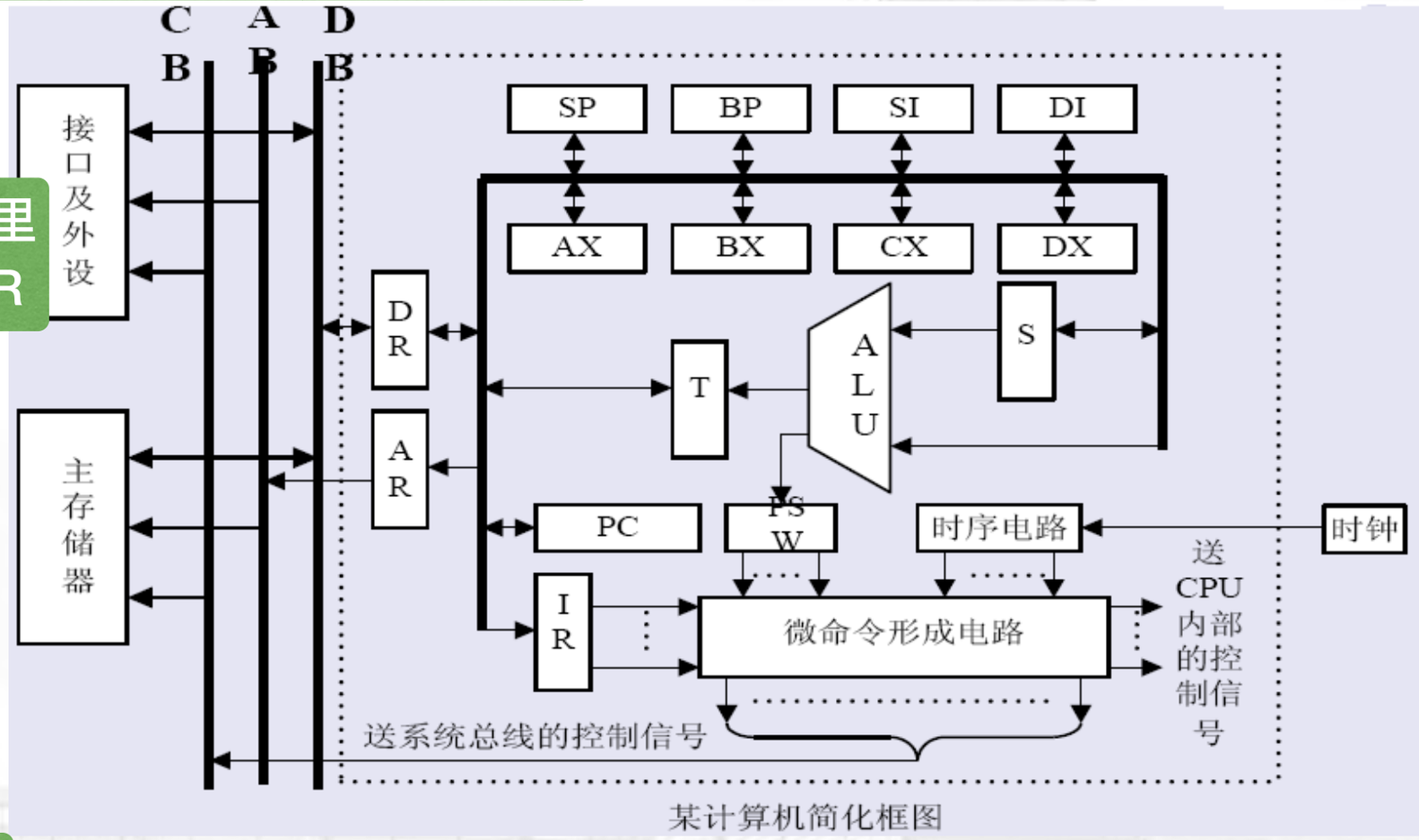
内存里的AR

内存里的DR

内存

把DS:[2000h]的数直接放入

IP指针自加1



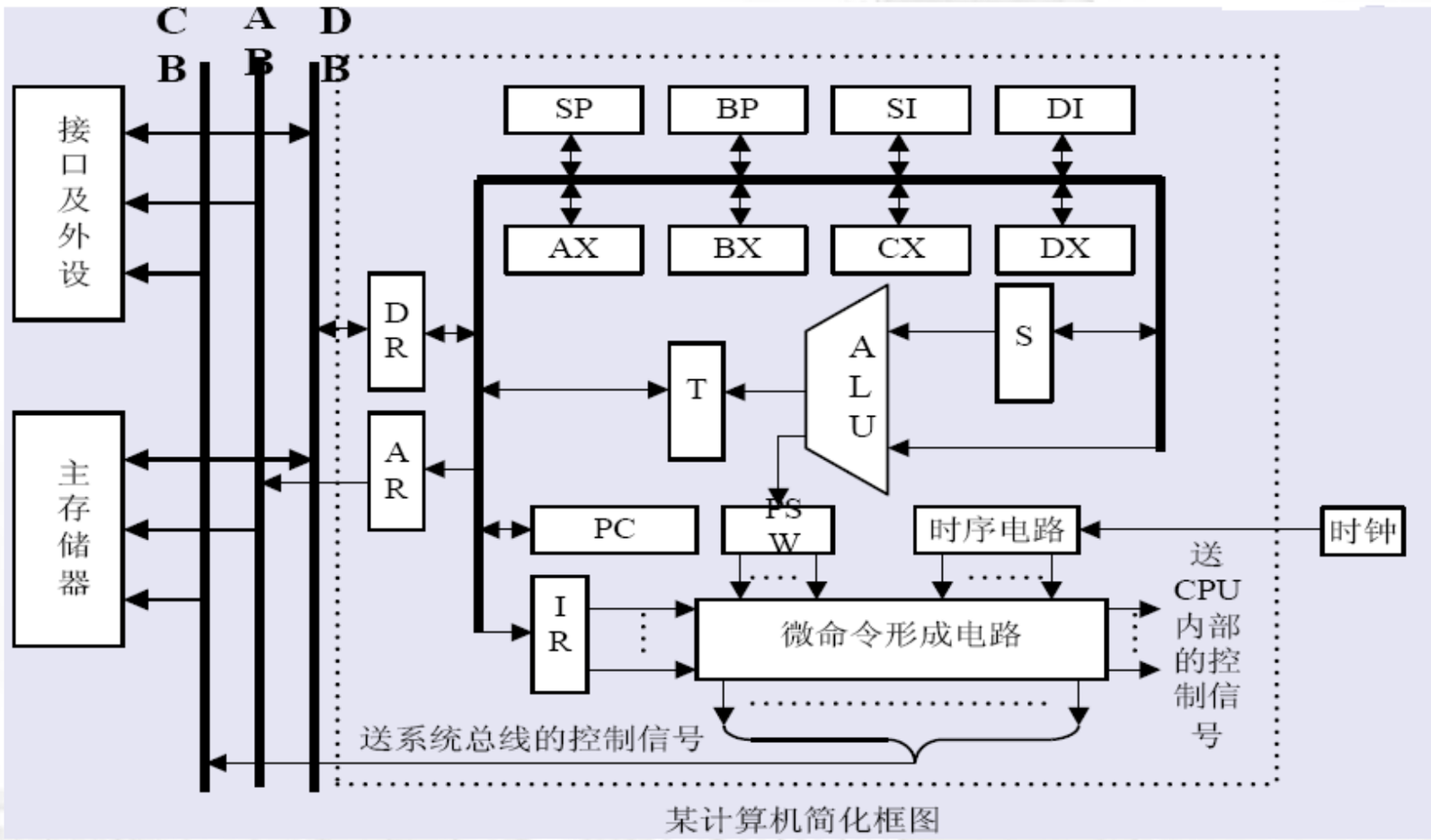
某计算机简化框图

深入CPU指令执行过程

以指令add [2000h], ax为例

- 执行Execute
  - AX->ALU
  - Add, ALU->T

加法结果放入T



某计算机简化框图

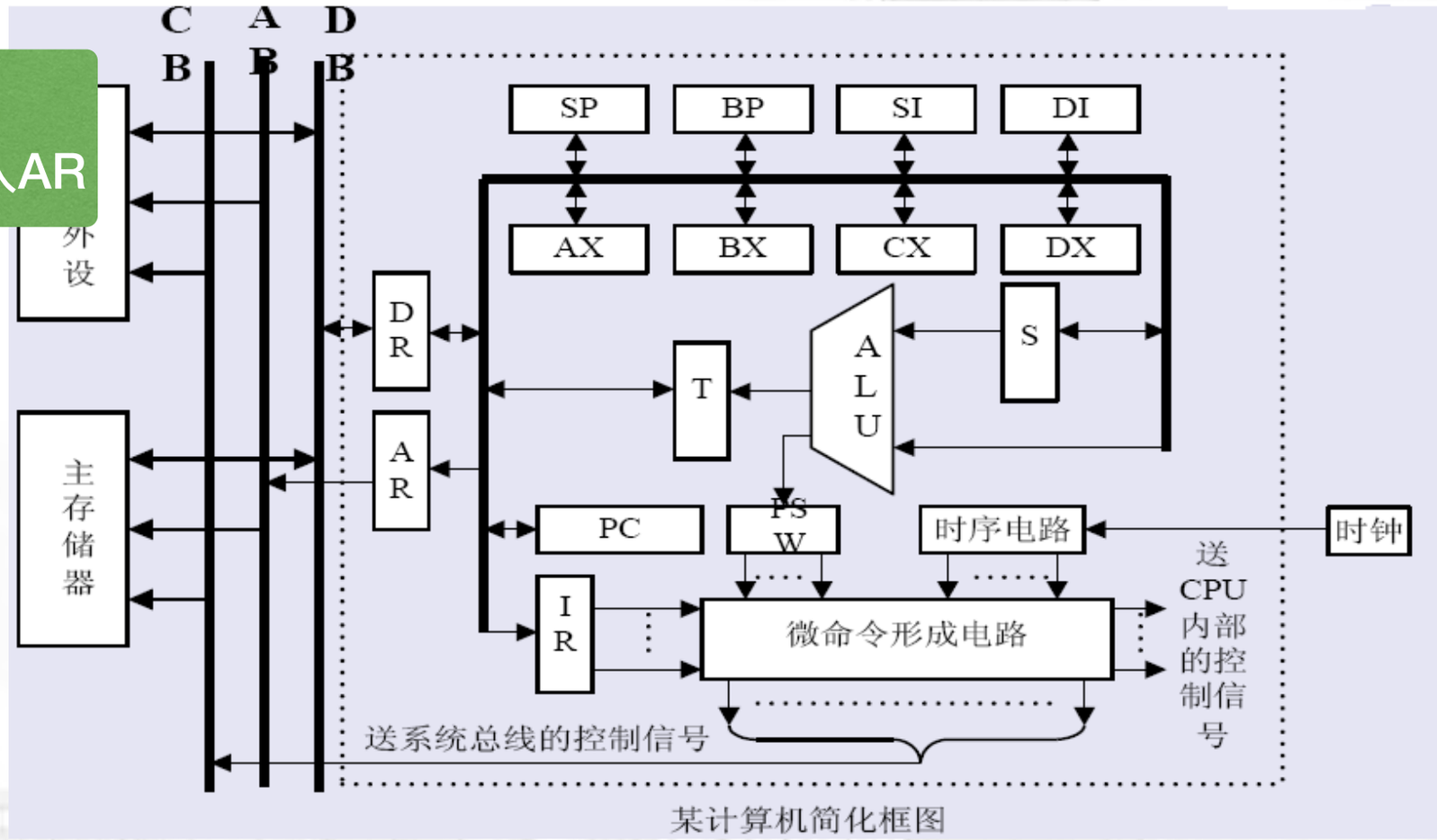
深入CPU指令执行过程

以指令add [2000h], ax为例

- 存储Store
  - T->DR
  - DR->DB
  - IR(2000h)->AR
  - AR->AB
  - DB->MDR
  - AB->MAR
  - Write

结果T送入DR

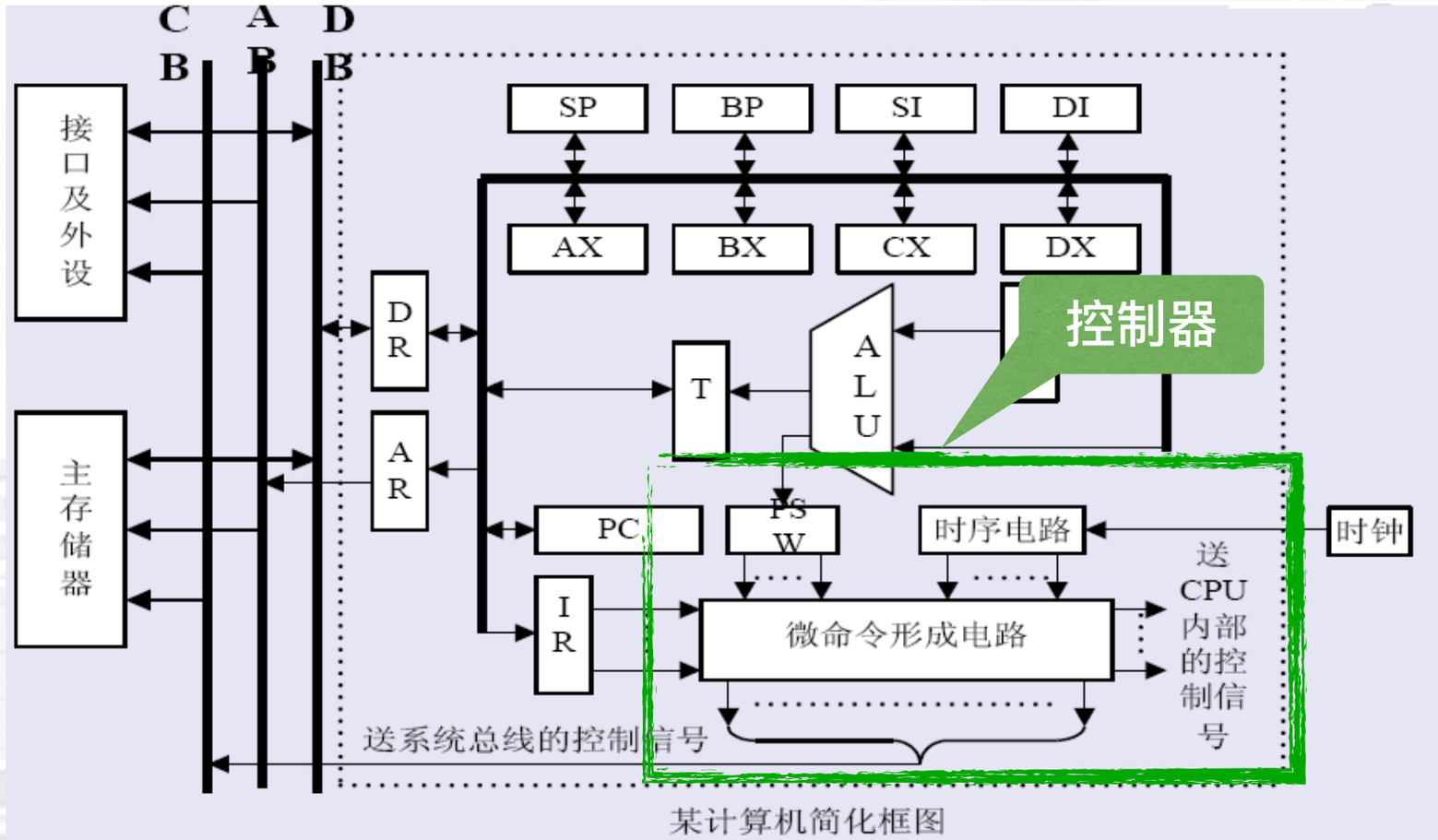
从IR中取出200000h，送入AR



某计算机简化框图

深入CPU指令执行过程

- 这个例子给我们一些insights:
  - 一条机器指令，大约由10-30个更基础的器件级的操作构成，called微操作
  - 微操作需要按照顺序进行（微操作需要时序控制）
- 微操作谁在控制?
  - CPU内的控制单元CU!



某计算机简化框图

关于时序与控制

- 时序系统的作用：将各种控制信号严格定时，在时间上相互配合完成某一功能。
- 时序信号通常划分为4级：
- 指令周期
- 机器周期（CPU周期）
- 时钟周期（节拍周期）
- [时钟脉冲、节拍脉冲]

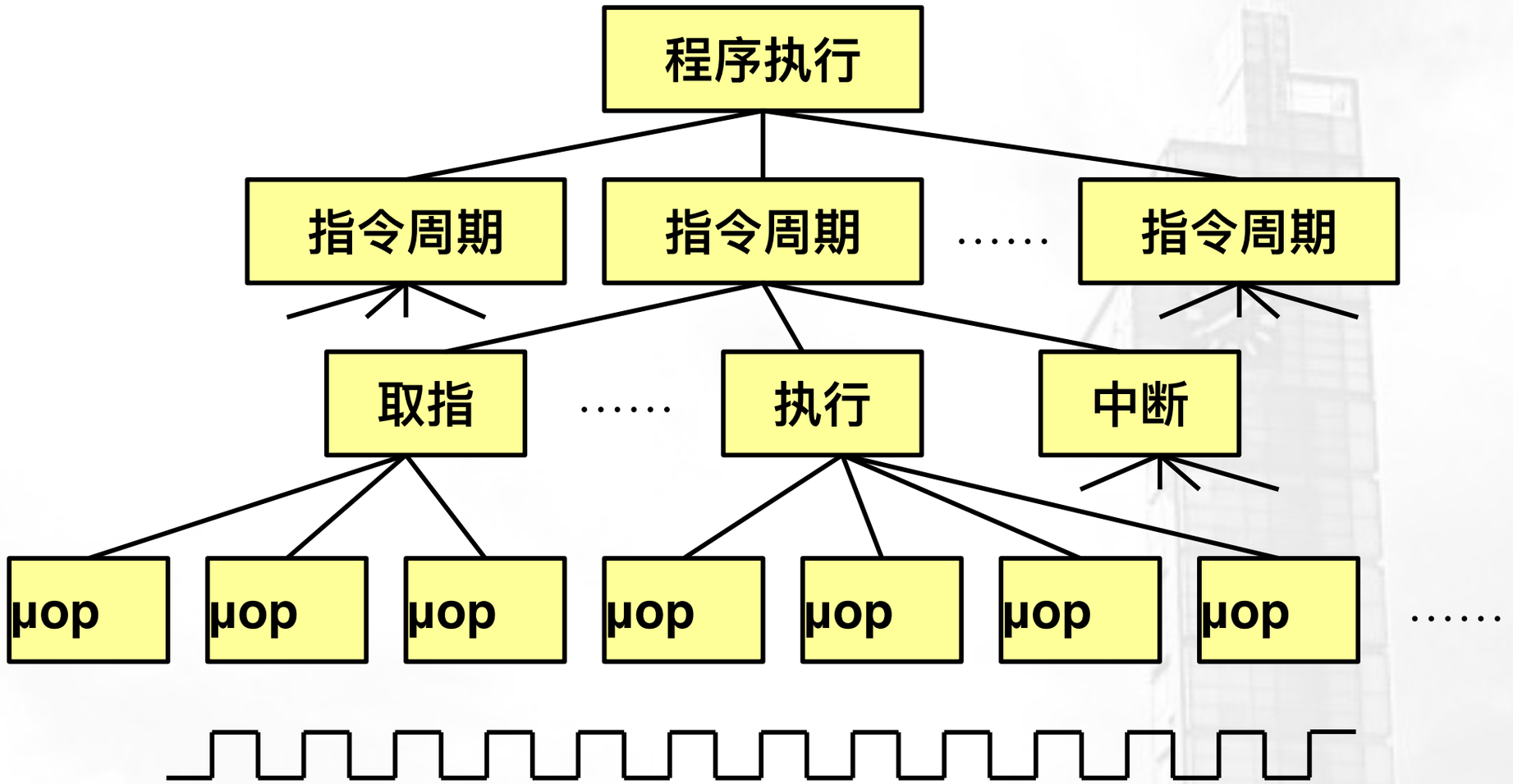
关于时序与控制

指令周期

CPU周期/机器周期

节拍周期

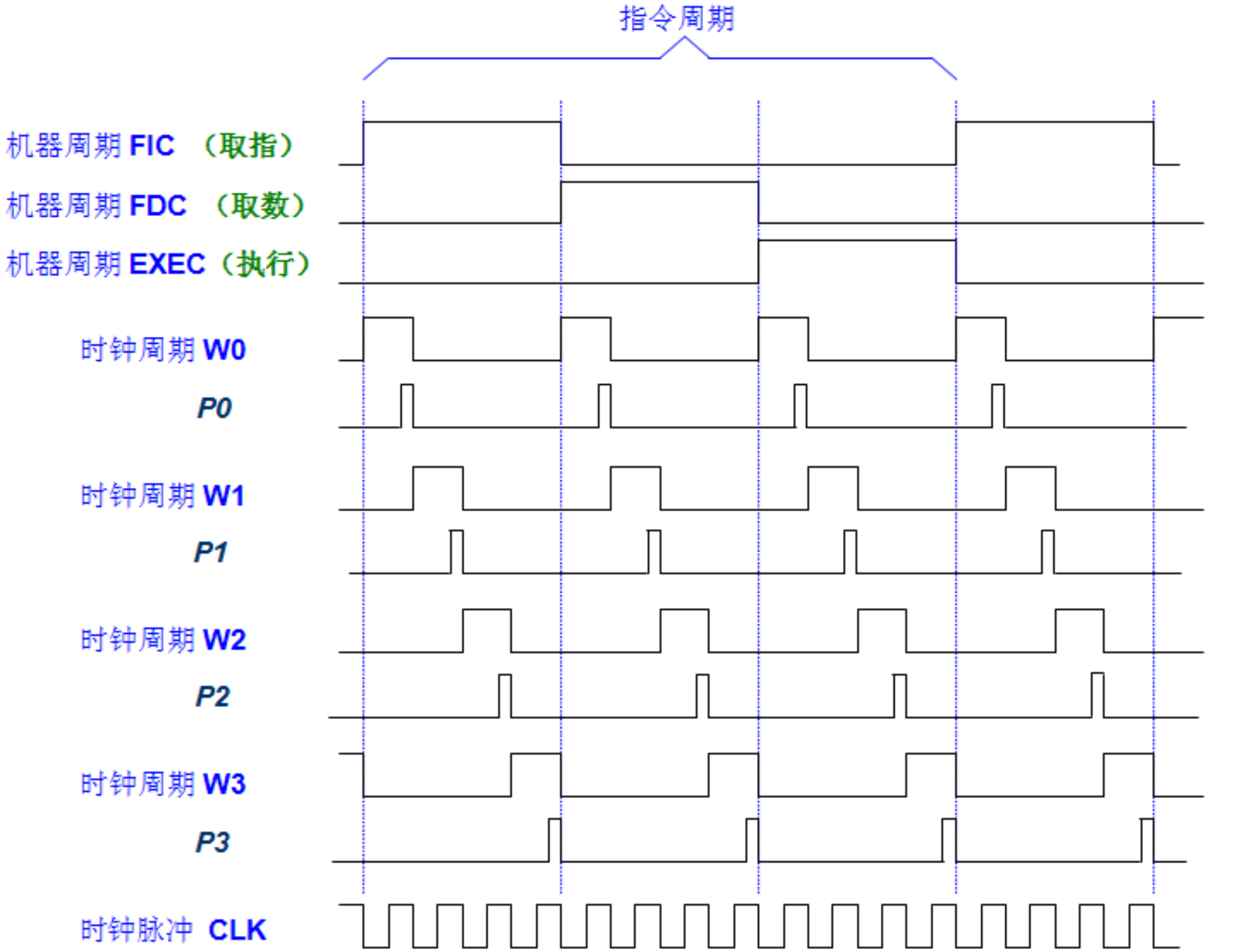
时钟脉冲



关于节拍周期：某种最复杂的微操作所需要的时间

关于CPU/机器周期：指令包含的微操作个数不等，不一定相同

关于时序与控制


指令周期
机器周期 FIC （取指）
机器周期 FDC （取数）
机器周期 EXEC（执行）
时钟周期 W0
P0
时钟周期 W1
P1
时钟周期 W2
P2
时钟周期 W3
P3
时钟脉冲 CLK

关于时序与控制

- 时序控制方式：
  - 同步、异步、联合
- 同步控制
  - 指令执行或指令中每个控制信号都由事先确定的统一的时序信号进行统一控制。
- 可能有具体差异：
  - 定长的机器周期，定长的指令周期
  - 定长的机器周期，变长的指令周期
  - 变长的机器周期，变长的指令周期
  - 折中方案

关于时序与控制

- **异步控制**

当控制器发出某一操作控制信号后，等待执行部件完成操作后发回“回答(ACK)”信号，再开始新的操作。没有统一的时钟对信号进行同步。每条指令的指令周期可由多少不等的机器周期数组成

- **联合控制** **同步控制和异步控制相结合的方式。**

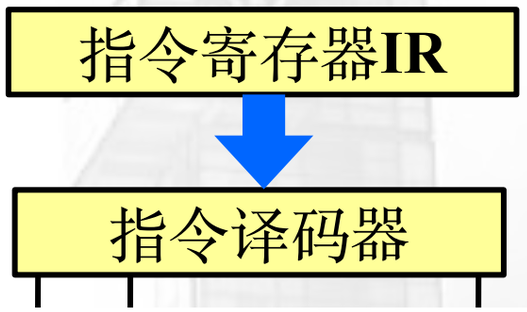
大部分微操作序列安排在固定的机器周期中，对某些时间难以确定的操作则采用“应答”方式。

如：CPU访问存储器时，依靠其送来的“READY”信号作为读 / 写周期的结束。

关于时序与控制

总结：指令执行过程涉及三种时序信号（或四种）

- 指令周期：执行一条指令所用的时间
- CPU周期：完成一个子周期（一组微操作）所用的时间
- 节拍周期：完成一个微操作所用的时间
- （时钟脉冲：微操作过程中的最小动作单位）



Then, why timing matters ?

CPU执行程序有严格的时间顺序性，通常利用时序电路为控制器提供所需的时序信号。最基本的时序信号为节拍，它由节拍脉冲发生器产生。

